

The Dangers of Change Approval Processes

Jose Velazquez Saenz

Bellevue University

CSD380: DevOps

Professor Sue Sampson

July 11, 2026

The Dangers of Change Approval Processes

Introduction

Organizations use change approval processes to reduce the risk of software failures, security problems, and service interruptions. In many traditional technology environments, a proposed production change must be documented, reviewed by management, and approved by an advisory board. Although this process is intended to protect systems, an overly complicated approval structure can create new risks. It can slow delivery, encourage larger changes, reduce employee responsibility, and create a false sense of security. The DevOps Handbook explains that reliable software delivery should depend on fast feedback, automated testing, small changes, and shared responsibility rather than lengthy manual approvals. DevOps does not recommend eliminating change management. Instead, it encourages organizations to embed controls directly into their development and deployment processes.

Traditional Change Approval Processes

A traditional change approval process begins when an employee submits a request describing the proposed change, its expected impact, testing results, implementation plan, and rollback procedure. A change manager then decides whether to approve, revise, delay, or reject the change. This type of review can be valuable for major changes, legal decisions, and high-risk activities. However, problems occur when every change must follow the same process. A small application update may be reviewed in the same way as a major data-center migration. When all changes require meetings and manual approval, the process becomes a bottleneck. Manual approval in a deployment pipeline pauses the release until an authorized person approves or rejects it. This control may be useful for important changes, but applying it to every deployment adds waiting time without guaranteeing that the software is safer.

Slower Delivery and Larger Changes

One of the greatest dangers of a long approval process is the lead time it creates. Developers may quickly complete and test a change, but then wait days for the next advisory meeting. The actual work may take only a few hours, while approval becomes the longest part of the delivery process. Reducing heavyweight external approvals can negatively affect software delivery performance. Instead, automated testing, continuous integration, and monitoring should be used. These practices allow problems to be discovered earlier by people who understand the code and the system.

Frequent approval requirements can also encourage teams to combine multiple changes into a single large release. Employees may avoid the approval process by submitting fewer but larger requests. Large releases are harder to test, troubleshoot, and reverse. If a failure occurs, it may be difficult to know which change caused the problem. Smaller releases are safer because they limit the number of possible causes and are easier to roll back.

A False Sense of Security

Approval can be mistaken for proof that a change is safe. However, an approval only confirms that someone authorized the change. It does not prove that the software is correct, secure, or properly tested. Managers or board members may approve requests based on incomplete information or assume that another person completed the technical review.

Research has shown that external approvals did not improve change failure rates. This is important because organizations sometimes respond to failures by adding more forms, signatures, and approval levels. These controls may make the organization appear more cautious while failing to address the real causes of failure, such as weak testing, poor monitoring, or oversized releases. The process can also become focused on paperwork rather than on actual risk. Employees may learn how to complete the required forms without improving the quality of the change. A ticket and an approval record can support auditing, but they do not prove that meaningful testing or technical analysis occurred.

Reduced Ownership and Employee Frustration

DevOps promotes shared responsibility for building, deploying, and supporting software. A separate approval board can undermine this sense of ownership by leading employees to believe that responsibility shifts to the approver. Developers may assume that an advisory board is responsible for identifying risks, while they assume that the development team has already completed a detailed review.

This separation can create blame when a deployment fails. Developers may argue that the board approved the change, while managers may claim that the request lacked accurate information. A peer-review process creates more direct accountability because a qualified team member reviews the code, configuration, or deployment script.

Slow approval systems can also frustrate employees. Engineers may feel that they are not trusted to manage systems they understand. When employees believe the process adds little value, they may seek shortcuts, classify routine changes as emergencies, or make undocumented changes. These actions can create even greater operational and security risks. Instead of treating speed and control as opposing goals, it should be adapting change management to support DevOps. A good process should make the safest method the easiest method for employees to follow.

A Better Risk-Based Approach

Organizations should not eliminate all approvals. Instead, they should classify changes according to risk. Low-risk, frequently performed changes can be preapproved when they follow a tested, documented process. Higher-risk changes may require additional technical or management review, while emergency changes should follow a faster, but still documented, procedure.

Routine changes can be controlled through automated testing, security scanning, peer-reviewed code, version-controlled configurations, deployment records, monitoring, and tested rollback

procedures. These controls examine the actual change and can provide faster feedback than a committee meeting. Deployment gates are another procedure that should be used. These gates can verify quality, security, system health, and compliance before a release proceeds. Human approval can then be reserved for unusual changes involving serious concerns. An advisory board can still serve an important purpose, but its role should change. Instead of approving every ticket, it can establish policies, study failed changes, coordinate major events, identify repeated risks, and improve automated controls.

Conclusion

Change approval processes are designed to protect organizations, but excessive reliance on manual approvals can create serious dangers. Large processes increase lead times, encourage larger releases, weaken team ownership, frustrate employees, and create a false belief that approval guarantees safety.

A DevOps approach does not remove governance. It moves governance closer to the work and makes it continuous. Peer review, automated testing, monitoring, small deployments, and clear audit records can provide stronger protection than a committee reviewing a written request shortly before deployment. The best approach is a risk-based system in which routine changes are automated or pre-approved, knowledgeable peers review technical changes, and major risks receive additional oversight. These practices help organizations improve speed, reliability, and security simultaneously when controls are built into the delivery process.

References

Atlassian. (2026c). *It change management: ITIL framework & best practices* .

<https://www.atlassian.com/itsm/change-management>

Azure DevOps. (2022). *Control deployments with Gates and approvals - azure pipelines* . Microsoft learn.

<https://learn.microsoft.com/en-us/azure/devops/pipelines/release/deploy-using-approvals?view=azure-devops>

DORA. (2025, October 30). *Dora | Capabilities: Streamlining change approval*.

<https://dora.dev/capabilities/streamlining-change-approval/>

Kim, G., Humble, J., Debois, P., Willis, J., Forsgren, N., & Allspaw, J. (2021). *The devops handbook: How to create world-class agility, reliability, & Security in Technology Organizations*. IT Revolution Press.

OpenAI. (2026). ChatGPT (40 mimi). <https://chat.openai.com/chat>

Serna, A., Creenaune, J., Slade, P., & Islam, Z. (2020, May 12). *Best practices for change management in the age of devops* . inside Atlassian. <https://www.atlassian.com/blog/atlassian-engineering/best-practices-for-change-management-in-the-age-of-devops>